



Universidad de Costa Rica

Facultad de Ingeniería

Escuela de Ciencias Computación e Informática

ECCI

Escuela de
**Ciencias de la
Computación e
Informática**

Programación Paralela y Concurrente

Tarea Programada 1:

Simulación de Colonia de Hormigas Buscadoras de Alimento

Sasha Chavarría Arias C32078

Isa Palacios Vindas C41236

Justin Steve Burgos Muñoz C31361

Emanuel González Chaves C03361

Emanuel Hernández Arauz B93699

Arturo Alfaro Díaz

Prof. Sleyter Angulo

I Semestre, 2026

Introducción

A lo largo del último siglo, el desarrollo de la ingeniería computacional, se ha consolidado como un pilar fundamental para la resolución de problemas complejos en múltiples áreas de conocimiento. Inicialmente, la mejora en el rendimiento de los sistemas computacionales se sustentó en el aumento del número de transistores por circuito integrado, la Ley de Moore. Sin embargo, hacia finales del siglo XX, esta tendencia comenzó a enfrentar limitaciones físicas relacionadas con el consumo energético y la disipación de calor.

Ante estas restricciones, surgió un rediseño en hardware orientado en la incorporación de múltiples núcleos de procesamiento en un mismo sistema. Este enfoque permitió conocer y utilizar el paralelismo como estrategia para mejorar rendimiento, dando lugar al desarrollo de técnicas de programación concurrente, de esta forma ampliando las posibilidades computacionales.

En este contexto, el crecimiento ha facilitado la implementación de modelos de simulación cada vez más complejos, extendiendo su aplicación a disciplinas como las ciencias naturales. Entre estos modelos, las simulaciones basadas en comportamientos colectivos, como el de las colonias de hormigas resultan particularmente relevantes debido a su capacidad para representar dinámicas emergentes a partir de interacciones individuales simples.

El presente proyecto propone la simulación de una colonia de hormigas mediante el uso de hilos de ejecución (*threads*), donde cada hormiga actúa como una entidad independiente en la búsqueda de recursos compartidos. Asimismo, se evaluará el impacto del paralelismo en el tiempo de ejecución de la simulación. La importancia del presente proyecto radica en la interseccionalidad de áreas de estudio, trabajadas por medio de simulaciones.

Objetivos

Objetivo general

Desarrollar una simulación de hormigas por medio del paralelismo utilizando hilos de ejecución para modelar el comportamientos de las hormigas.

Objetivo específico

1. Modelar el comportamiento de una colonia de hormigas en un entorno simulado, donde cada hormiga sea representada como un hilo independiente de ejecución.
2. Medir el tiempo de ejecución de la simulación bajo diferentes condiciones de concurrencia, con el fin de evaluar el impacto del paralelismo en el rendimiento del sistema.
3. Registrar y reportar el total de alimento recolectado, así como los eventos relevantes durante la ejecución de la simulación.

Desarrollo

Descripción del programa

El presente proyecto consiste en la simulación de una colonia de hormigas mediante un modelo concurrente, en el cual cada hormiga es representada como un hilo de ejecución independiente. Cada una de ellas exploran un entorno compartido con el objetivo de localizar y recolectar alimento, representado a través de fuentes de recursos comunes.

Cuando una hormiga identifica una fuente de alimento, se registra el hallazgo y se disminuye la cantidad disponible en dicha fuente. Dado que múltiples hormigas pueden acceder simultáneamente a un mismo recurso, se generan potenciales condiciones de carrera que pueden comprometer la integridad de los datos. En consecuencia, resulta necesario implementar mecanismos de sincronización que regulen el acceso concurrente, específicamente mediante el uso de mutex para garantizar la exclusión mutua.

Esto fue desarrollado por medio de pthreads para la creación de hormigas y por medio de árboles para la búsqueda de alimento. Primeramente se crea una simulación del entorno, donde el árbol no solo funciona como estrategia algorítmica de recorrido sino como un árbol real con hojas que son utilizadas como el alimento para las hormigas. A partir de esto, las hormigas se encargan de la recolección de hojas, disminuyendo la cantidad de recursos disponibles.

En la recolección de hojas se presenta el mutex para evitar condiciones de carrera sobre dos hormigas llegando a una misma hoja, por lo que se bloquea la hoja encontrada por la hormiga mientras es cambiado su estado, ya sea la disminución de parte de la hoja o de plano su eliminación.

Finalmente, el sistema genera métricas relevantes, tales como el total de alimento recolectado y el tiempo total de ejecución, permitiendo evaluar el comportamiento y desempeño del modelo bajo condiciones de paralelismo.

Diagramas

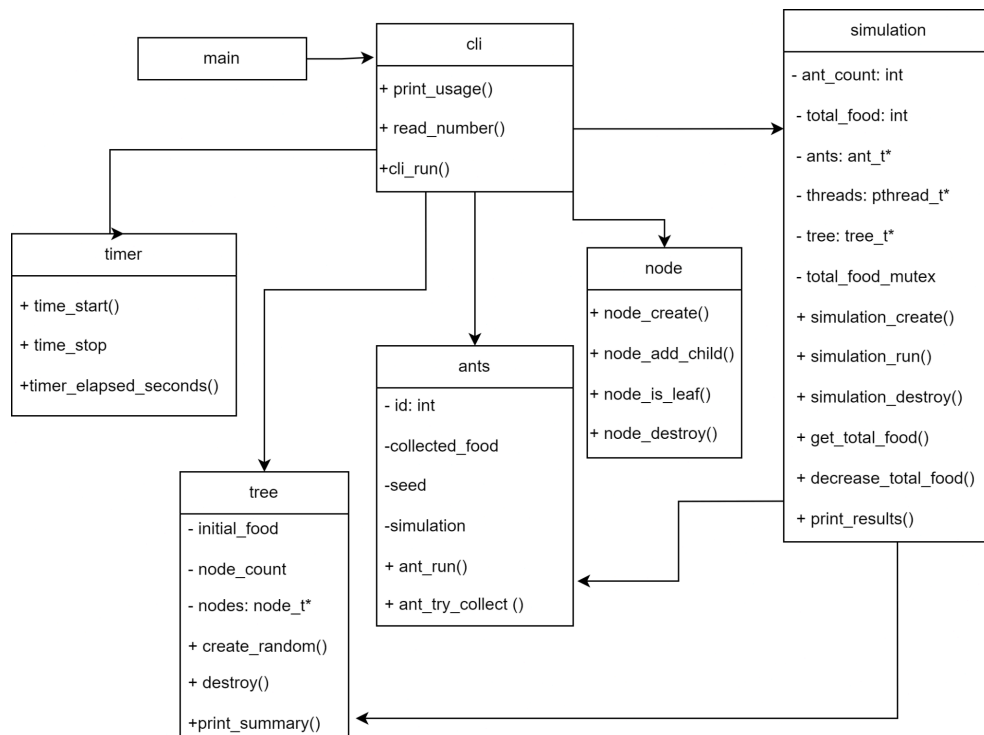


Figura 1. Diagrama de clases del proyecto, según la implementación creada.

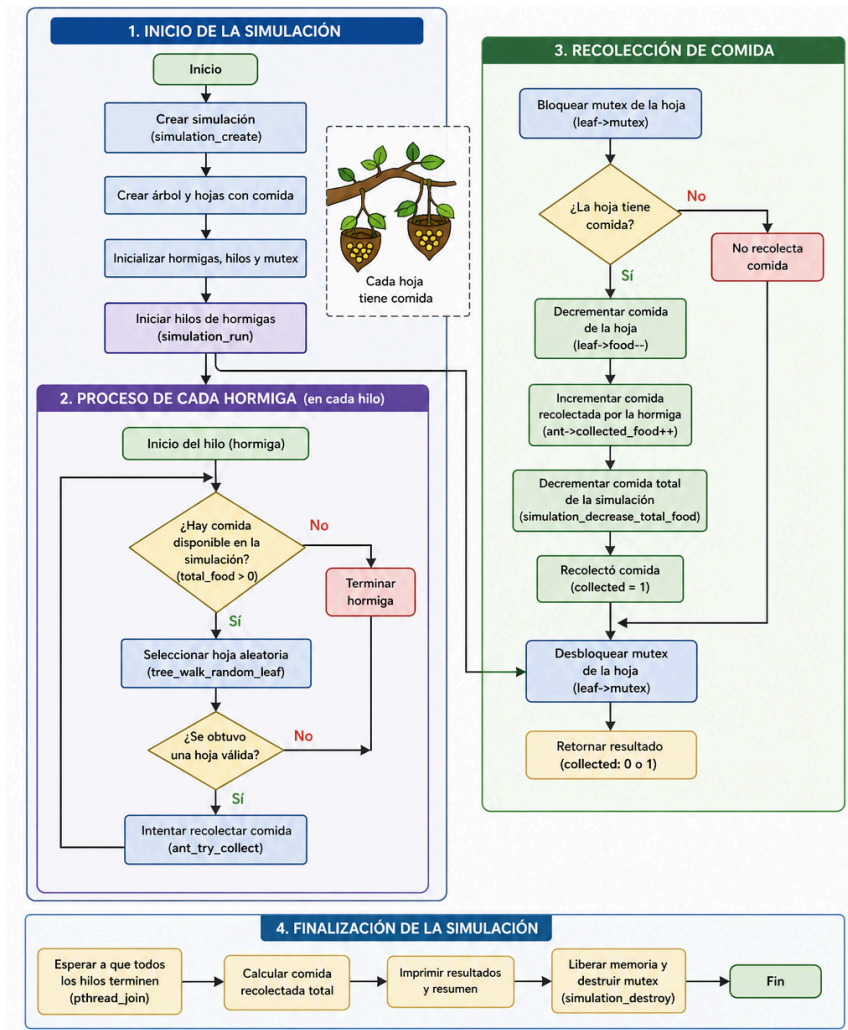


Figura 2. Diagrama de flujo explicando procesos fundamentales de la simulación. (OpenAI, 2026)

Investigación del tema

La programación concurrente constituye un enfoque fundamental en el desarrollo de software moderno, especialmente en sistemas que buscan aprovechar arquitecturas multinúcleo. La biblioteca POSIX Threads, permite la creación y gestión de múltiples hilos de ejecución dentro de un mismo proceso. A través de funciones como *pthread_create*, es posible definir flujos de ejecución independientes que comparten el mismo espacio de memoria, lo que facilita la cooperación entre tareas pero, al mismo tiempo, introduce retos asociados al acceso concurrente a datos compartidos (The Open Group, 2018).

Cuando múltiples hilos acceden a una misma variable o estructura de datos sin un control adecuado, pueden surgir las denominadas condiciones de carrera (race conditions). Este fenómeno ocurre cuando el resultado de una operación depende del orden en que se ejecutan las instrucciones de los hilos, lo cual puede variar de una ejecución a otra. Como señalan Abraham Silberschatz, y compañía (2018),

las condiciones de carrera representan uno de los principales problemas en sistemas concurrentes, ya que pueden generar resultados inconsistentes y difíciles de reproducir.

Para mitigar estos problemas, es necesario implementar mecanismos de sincronización que regulen el acceso a los recursos compartidos. Uno de los más comunes es el mutex, basado en el principio de Exclusión mutua, el cual establece que solo un hilo puede acceder a una sección crítica del código en un momento dado. En pthreads, los mutex se gestionan mediante funciones como *pthread_mutex_lock* y *pthread_mutex_unlock*, que permiten bloquear y liberar el acceso a un recurso compartido, respectivamente (The Open Group, 2018).

Solución

Distintas maneras de resolver el problema (comparativa de enfoques).

Arquitectura

El sistema se estructura en varios módulos principales que colaboran entre sí para modelar la simulación concurrente:

- **cli.h / main.h:** Encargado de la lectura de parámetros desde la línea de comandos y de iniciar la simulación.
- **simulation.h:** Controla la lógica general del sistema, incluyendo la creación de hilos, ejecución y recolección de resultados.
- **ant.h:** Define el comportamiento individual de cada hormiga como hilo independiente.
- **tree.h:** Modela el entorno mediante una estructura de árbol donde se encuentran las fuentes de alimento.
- **node.h:** Representa cada nodo del árbol, incluyendo su mutex para sincronización.
- **timer.h:** Permite medir el tiempo total de ejecución.

Cada hormiga es representada como un hilo (pthread), el cual interactúa con un entorno compartido (árbol), accediendo a recursos protegidos mediante mecanismos de sincronización.

Flujo del programa

El funcionamiento del sistema sigue los siguientes pasos:

1. El programa inicia desde la terminal con parámetros definidos por el usuario.
2. Se validan los datos de entrada (cantidad de hormigas, nodos, etc.).
3. Se crea la simulación (*simulation_create*):
 - a. Se inicializa el árbol de nodos.
 - b. Se asigna comida a las hojas.
 - c. Se crean las estructuras de hormigas.
 - d. Se inicializan mutex globales.

4. Se inicia el temporizador.
5. Se crean múltiples hilos, uno por cada hormiga (`pthread_create`).
6. Cada hormiga ejecuta la función `ant_run`, donde:
 - a. Recorre el árbol aleatoriamente.
 - b. Encuentra hojas con comida.
 - c. Intenta recolectar alimento.
7. Para acceder a la comida:
 - a. Se bloquea el mutex del nodo hoja.
 - b. Se modifica el recurso compartido.
 - c. Se desbloquea el mutex.
8. Se actualiza la comida total con mutex global.
9. Los hilos finalizan cuando no queda comida.
10. El programa espera a todos los hilos (`pthread_join`).
11. Se detiene el temporizador.
12. Se imprimen los resultados finales.

Estructuras de datos utilizadas

- **Hormiga (`ant_t`)**

Representa un hilo independiente.

Incluye:

1. ID de la hormiga
2. Cantidad de comida recolectada
3. Semilla para aleatoriedad
4. Referencia a la simulación

- **Nodo(`node_t`)**

Representa una posición en el entorno.

Incluye:

1. ID
2. Cantidad de comida
3. Hijos
4. Mutex (para sincronización)

- **Arbol(`Arbol_t`)**

Representa una posición en el entorno.

Incluye:

1. Nodo raíz
2. Lista de nodos

3. Cantidad de hojas
4. Comida total inicial

- **Simulacion(Simulacion_t)**

Representa una posición en el entorno.

Incluye:

1. Arreglo de hormigas
2. Arreglo de hilos
3. Árbol
4. Comida total
5. Mutex global

Enfoque de solución problema Condición de Carrera

Al tener múltiples hormigas buscando comida simultáneamente se pueden generar las siguientes inconsistencias:

- Dos hormigas encuentran la misma comida al mismo tiempo.
- Dos hormigas pueden intentar modificar el número de alimento encontrado y dar como resultado un error.

Dichos problemas se pueden solucionar con el uso de Mutex para este problema específico se aplicó de la siguiente manera:

- `pthread_mutex_lock(&leaf->mutex);`

Evita que múltiples hormigas modifiquen la misma hoja al mismo tiempo.

- `pthread_mutex_lock(&simulation->total_food_mutex);`

Protege la variable global *total_food*.

Casos de prueba

Se realizaron las pruebas con distintos números de hormigas para poder denotar el impacto de tener un mayor cantidad de procesos funcionando en paralelo.

Caso	Parámetros	Comida inicial	Recolectada	Consistencia
Small	4 30 3 5	95	95	✓OK
Medium	16 500 3 10	3320	3320	✓OK
Large	64 5000 3 20	66640	66640	✓OK

Conclusiones

El desarrollo del presente proyecto permitió evidenciar la aplicabilidad del paralelismo como estrategia para la resolución de problemas computacionales complejos, particularmente en el modelado de sistemas basados en comportamientos colectivos. A través del desarrollo de una simulación de una colonia de hormigas utilizando hilos de ejecución, se logró representar de manera efectiva la dinámica generada por la interacción de múltiples entidades independientes que compiten por recursos compartidos.

Se cumplieron satisfactoriamente los objetivos al desarrollar un sistema funcional que modela el comportamiento de las hormigas bajo un enfoque concurrente, donde cada hilo representa una unidad autónoma de ejecución. También, la medición del tiempo de ejecución bajo condiciones concurrentes permitió analizar el impacto del paralelismo en el rendimiento del sistema, destacando mejoras en eficiencia frente a enfoques secuenciales, así como la importancia de una adecuada gestión de recursos compartidos. Finalmente, se cumplió con el registro y reporte de resultados, incluyendo la cantidad total de alimento recolectado y la verificación de consistencia de la simulación, lo cual garantiza la validez de los datos obtenidos.

Adicionalmente, el uso de mecanismos de sincronización, como los mutex, resultó fundamental para evitar condiciones de carrera y asegurar la integridad de los recursos compartidos, lo que refuerza la importancia de las técnicas de control de concurrencia en sistemas paralelos.

En síntesis, el proyecto no solo cumplió con los objetivos planteados, sino que también permitió consolidar conocimientos teóricos sobre programación concurrente mediante su aplicación práctica, evidenciando cómo el paralelismo puede ser aprovechado para modelar fenómenos complejos de manera eficiente y segura.

Referencias bibliográficas

- Abraham Silberschatz, Peter B. Galvin, & Greg Gagne. (2018). *Operating System Concepts* (10th ed.). Wiley.
- OpenAI. (2026). *Diagrama de flujo – proceso de las hormigas y su comida* [Imagen generada por IA]. ChatGPT (DALL·E).
- The Open Group. (2018). *Base Specifications Issue 7, IEEE Std 1003.1*.